Departamento de Electrónica, Telecomunicações e Informática da Universidade de Aveiro

Algoritmos e Estruturas de Dados

2026/2027 — 1º Semestre

1º Trabalho — Gerador de Cenas 2D (TAD Primitive e TAD VectorScene)

Data limite de entrega: 20 de novembro de 2026, às 18 horas



Objetivos

Este trabalho tem como objetivos principais:

1. **Concluir o desenvolvimento e testar os TADs Primitive e VectorScene,** que permitem instanciar cenas 2D compostas por formas geométricas (i.e., primitivas) heterogéneas (Círculos, Retângulos, Segmentos de Reta e Polígonos), com suporte para transparência (*Alpha*) e camadas (*layers*) de profundidade (*Z-Index*), e gerar as correspondentes imagens num formato simples.

2. **Analisar a eficiência e a complexidade computacional de algoritmos** para **(1)** ordenação das primitivas de acordo com a sua profundidade, **(2)** seleção espacial (*Picking*) otimizados com caixas delimitadoras (*Axis-Aligned Bounding Boxes* – AABB), com medição do número de operações.

3. **Criar uma composição artística original gerada pelo software desenvolvido**.

Conceitos Fundamentais de Computação Gráfica 2D

1. **Camadas (*Layers*) e Ordenação por Profundidade (*Painter's Algorithm*):**
   Cada primitiva geométrica possui um valor inteiro de profundidade (**z_index**). A geração da imagem final da cena (renderização) segue o *Algoritmo do Pintor*, no qual os objetos são desenhados ordenadamente do fundo para a frente: desenham-se primeiro as primitivas com menor **z_index** e sobrepõem-se progressivamente os objetos com maior **z_index**.
2. **Transparência e Mistura de Cores (*Alpha Blending*):**
   O parâmetro de opacidade (***α in [0.0, 1.0]***) determina o grau de transparência de uma primitiva. Durante a rasterização de um píxel, a cor final resulta da combinação linear entre a cor da nova primitiva (***C_nova***) e a cor já existente no *canvas* (***C_fundo***), segundo uma simples interpolação linear para cada canal RGB (***C_final = α C_nova + (1 −α) C_fundo***).

3. **Renderização e Rasterização de Primitivas:**
   O processo de renderização de cada primitiva converte formas vetoriais contínuas numa grelha discreta de píxeis (*raster image*). Para otimizar o processo, limita-se a amostragem de píxeis à janela correspondente à caixa delimitadora (*AABB*) da primitiva, testando apenas se as coordenadas de cada um dos píxeis pertencem ao interior geométrico da forma.
4. **Seleção Espacial em Duas Fases (*Two-Phase Picking*):**
   Para identificar a primitiva de topo visível num determinado ponto da imagem final, aplica-se um filtro hierárquico em duas fases: a **Fase Inicial menos precisa (*Broad-phase*)** realiza um teste rápido e de baixo custo computacional sobre a caixa delimitadora (AABB) de cada objeto; apenas para as AABBs que contêm esse ponto, a **Fase Secundária (*Narrow-phase*)** executa o teste exato de pertença geométrica.

Desenvolver os TADs Primitive e VectorScene

Com base nos ficheiros de interface (**primitive.h** e **vector_scene.h**) e de implementação parcial (**primitive.c** e **vector_scene.c**) fornecidos, deve:

1. **Analisar com cuidado os ficheiros**, compreendendo a representação de primitivas usando uma **União Etiquetada (*Tagged Union*)** no **TAD Primitive**, a representação das primitivas que definem uma cena no **TAD VectorScene** e a gestão dos vértices que definem um polígono usando um array dinâmico de pontos (**Point* vertices**).

2. **Concluir o desenvolvimento das funções especificadas** nos ficheiros de interface e que ainda se encontrem incompletas nos ficheiros de implementação.

3. **Garantir a ausência total de fugas de memória (*Memory Leaks*)**: Toda a memória alocada dinamicamente (**malloc/realloc**) deve ser rigorosamente libertada (**free**). As submissões serão testadas com ferramentas de análise estática e dinâmica.

4. **Testar todas as funções desenvolvidas,** recorrendo aos programas e ficheiros de teste fornecidos e criando testes próprios.

Atenção:

- **Os ficheiros de interface (.h) especificam as funções dos TADs e NÃO podem ser alterados.**
- Para cada função são indicadas as pré-condições, pós-condições e regras de posse de memória, que devem ser obrigatoriamente tidas em conta.
- Caso facilite o desenvolvimento, poderão ser definidas e implementadas funções auxiliares adicionais (**static**) nos ficheiros **.c**, assinalando-as com comentários apropriados.
- O **código** desenvolvido deverá ser **claro** e **comentado**: os identificadores escolhidos para as variáveis e a estrutura do código, bem como os eventuais comentários, deverão ser suficientes para a sua compreensão.
- Não é necessário entregar qualquer relatório detalhado relativo ao desenvolvimento dos TADs.
- Serão avaliados os ficheiros de implementação **primitive.c** e **vector_scene.c**, juntamente com a composição do grupo (ficheiro **.vsc** e imagem gerada) e o relatório do trabalho.
- Deverão ser entregues todos os ficheiros (**.h**, **.c** e **Makefile**) do projeto.

Analisar a Eficiência da Ordenação por Profundidade (vector_scene_sort_by_z)

Após concluir o desenvolvimento do TAD, deve analisar a eficiência computacional do processo de ordenação por camada (**Z-Index**). Para isso deve:

1. Implementar e comparar duas estratégias de ordenação do array de primitivas da cena, registando as operações nos contadores da **struct SortStats**:
   Estratégia Básica: Algoritmo de complexidade O(N^2) – **Insertion Sort**
   Estratégia Otimizada: Algoritmo de complexidade O(NlogN) – **Merge Sort**

2. Realizar uma sequência de testes computacionais com cenas contendo um número crescente de primitivas (N de 100 a 100.000). Registar os tempos de execução e os valores dos contadores de comparações/trocas obtidos na **struct SortStats**.

3. Efetuar uma análise formal da complexidade dos dois algoritmos e comparar os resultados da análise teórica com os dados experimentais obtidos.

Analisar a Eficiência da Seleção Espacial (*Picking*)

Deve analisar a eficiência computacional do processo de seleção espacial (*Picking*) comparando:

1. Uma **estratégia direta** de verificação exata de pertença, para cada primitiva da cena.
2. A **estratégia otimizada** (em duas fases) com caixas delimitadoras (*Axis-Aligned Bounding Boxes* – AABB).

Registe nos contadores da **struct PickStats** as **operações realizadas** por essas duas estratégias ao efetuar **M operações de seleção em cenas com N objetos.** Determine **rácios de eficiência.**

Composição Artística Livre do Grupo

Cada grupo deve criar e submeter uma **composição artística original (art_group_XX_YY.vsc)** que demonstre todas as funcionalidades do gerador de cenas vetoriais (utilização de círculos, retângulos, segmentos, polígonos com múltiplos vértices, transparências e várias camadas de profundidade).

- A entrega deve incluir o **ficheiro de texto .vsc** e a respetiva **imagem final gerada.**

- **A**s composições mais criativas serão destacadas na galeria do projeto.

Escrita do Relatório (máx. 6 págs.)

O relatório deve ser claro e conciso (no máximo 6 páginas), incluindo obrigatoriamente:

1. **Introdução e Resumo da Arquitetura:** Breve descrição das opções de implementação adotadas nos **TADs Primitive** e **VectorScene**.
2. **Análise dos Algoritmos de Ordenação por Profundidade,** incluindo tabelas com os resultados dos testes (comparando tempos, comparações e trocas realizadas pelos dois algoritmos), gráficos ilustrativos e análise formal.
3. **Análise das Estratégias de *Picking*,** incluindo dados experimentais baseados nos valores dos contadores da **struct PickStats** e discussão sobre o ganho de desempenho.

4. **Composição Artística:** Imagem final obtida e breve descrição da cena criada pelo grupo.
5. **Conclusão:** Síntese dos resultados e considerações finais.

Critérios de Avaliação

- **Desenvolvimento e teste do código pedido (45%)**
  - Qualidade, clareza, estrutura do código e comentários.
  - Verificação do funcionamento correto e ausência total de fugas de memória validada por **valgrind**.
- **Relatório (45%)**
  - Aspetos gerais, clareza de apresentação e conclusões.
  - Análise experimental e formal dos algoritmos de ordenação por profundidade.
  - Análise das estratégias de seleção espacial.
- **Colaboração na Avaliação entre Pares (2 trabalhos) (10%)**
  - Testar o funcionamento do código de colegas, avaliar a sua qualidade e clareza de forma fundamentada e crítica.

Nota final do trabalho: A **nota atribuída ao desenvolvimento e teste das funções pedidas** será calculada pela média ponderada da classificação atribuída pelo docente e da classificação média atribuída pelos colegas (avaliação pelos pares, cada trabalho será avaliado por 3 a 4 estudantes).

A avaliação pelos pares está sujeita a validação pelos docentes, podendo ser descartada em casos de manifesta incoerência.

Atenção:

- O trabalho deve ser realizado em **grupos de 2 alunos**.
- Poderão vir a ser disponibilizados **ficheiros adicionais** no **Moodle**, com testes unitários automáticos e geradores de cenas, para poderem testar o vosso código.
- A entrega do trabalho será feita através do Moodle.
- Após a conclusão das submissões, cada estudante receberá a indicação dos dois trabalhos de colegas que deverá rever. Essa tarefa é parte integrante deste trabalho.